RetroQuest Compiler Construction

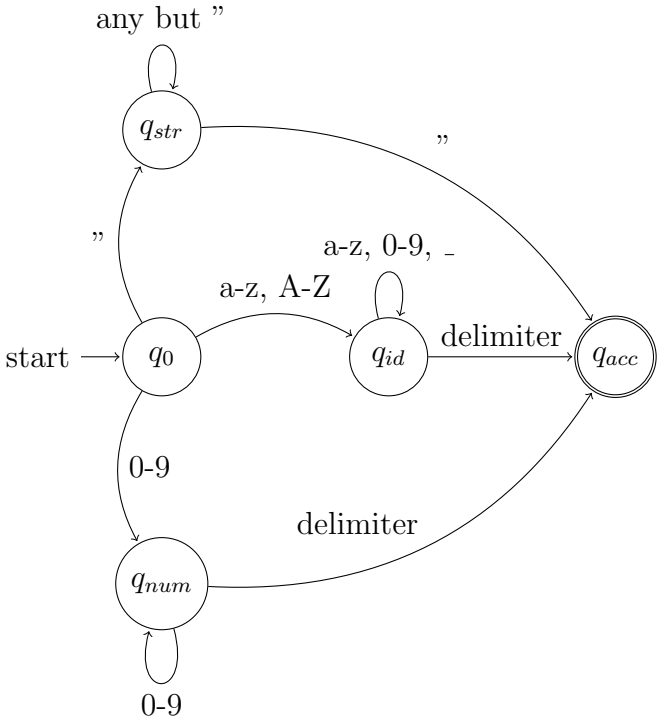
Handwritten Documentation

Hamadullah Arain, Tashkeel Pasha, Yousaf Bhatti

Spring 2026

1 Lexical Analyzer Design: DFA Diagram

The following DFA (Deterministic Finite Automaton) describes the transition logic for identifying key tokens in the RetroQuest DSL (Identifiers, Strings, and Numbers).

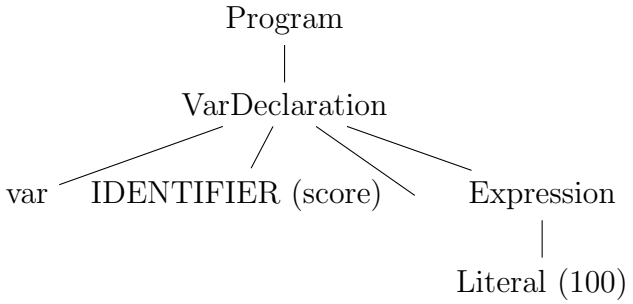


2 Parser Design: Parse Trees

Below are two complete parse trees representing the syntactic structure of core Retro-Quest statements.

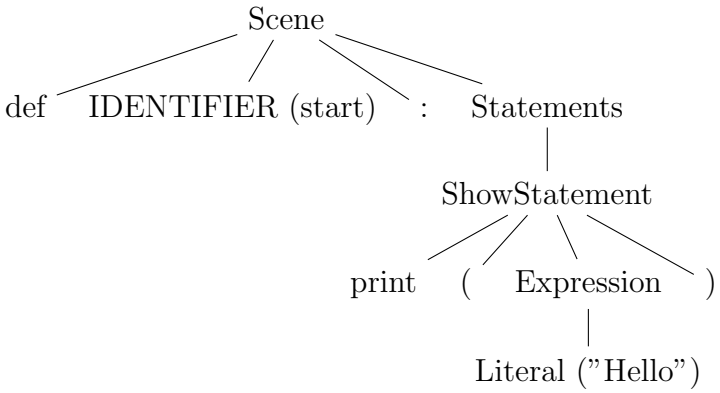
2.1 Parse Tree 1: Variable Declaration

Input: `var score = 100`



2.2 Parse Tree 2: Scene with Show Statement

Input: `scene start:  print("Hello")`



3 Semantic Analyzer: Symbol Table Design

The following table demonstrates how the Semantic Analyzer tracks scopes and type information for variables and scenes.

Identifier	**Type**	**Scope**	**Initial Value / Target**
score	Integer	Global	0
health	Integer	Global	100
is_alive	Boolean	Global	true
start	Scene	Global	Code Offset 0x00
hallway	Scene	Global	Code Offset 0x42
ending	Scene	Global	Code Offset 0x99

Table 1: Global Symbol Table for RetroQuest Compiler

3.1 Semantic Rules Applied

- **Type Consistency**: Arithmetic operations ($+$, $-$, $*$, $/$) are only permitted on identifiers marked as *Integer*.
- **Declaration Check**: Any *IDENTIFIER* used in a `goto` or `choice` must exist in the Symbol Table with type *Scene*.
- **Unique Labels**: Scene names must be unique within the global scope to prevent jump ambiguity.